



UMS
UNIVERSITI MALAYSIA SABAH

KP14203 OBJECT ORIENTED PROGRAMMING
SEMESTER 2, 2022/2023

LECTURER: DR. TAN SOO FUN

GROUP PROJECT REPORT
INVENTORY MANAGEMENT SYSTEM

PREPARED BY:

BIL	Name	MatricNo	Signature
1	SYAZA SYAZANA BINTI ANWAR		
2	SHWEHTTA A/P PARANI KUMAR		
3	LEENA A/P SELLAMANY		
4	NORSYLAH BINTI SAFRUDIN		
5	WAN NURADRIANA BALQIS BINTI WAN AZMI		

1.0 Project Abstract

An inventory management system is essential for companies that handle consumer product transactions because it allows for quality assurance. A large retail establishment could run out of a necessary item if inventory is not properly maintained. An efficient inventory management system will alert the retailer when it is time for a restock. The automatic tracking of large shipments is another benefit of using this inventory management system. Hand counting each pair of stocks will almost probably yield an error. By implementing an automated inventory management system, the danger of human error can be decreased.

The tracking of retail product theft is made much easier by an inventory management system, which also provides relevant data about store earnings and the need for theft-prevention tools. The list of things that need to be retrieved for packaging and shipping can also be included in the purchase order. The Inventory Management System is capable of a variety of duties in this scenario. It can help a warehouse worker locate items on an order list, encode shipping details like tracking numbers and delivery addresses, and remove purchased items from the inventory tally to keep an accurate count of the items that are still in stock. Together, all of this data enables businesses to track their inventory at all times. Real-time inventory information is straightforward to locate and look at thanks to simple database searches.

TABLE OF CONTENT

CONTENT	PAGE
Cover Page	1
1.0 Project Abstract	2
2.0 Introduction	4
2.1 Problem Statement	5
2.2 Objectives	5
2.3 Scope	6
2.4 Project Timeline	6
3.0 System Analysis and Design	7-9
3.1 Narrative Story	7
3.2 UML Diagram- Class Diagram	8-9
4.0 Implementation	10
5.0 Conclusion	11
6.0 Reference	12
7.0 Appendix	13

2.0 Introduction

This report provides an overview of the inventory management system. This inventory management system includes descriptions about all the modules in this system. The modules include the user login, Stock management, Order management, Warehouse management and the Revenue and expenses. This system provides the flow on how the inventory is going to be managed which starts from the user to the supplier. Moreover, to talk about this system, this inventory management system is a software solution designed to optimize the process of managing and tracking inventory, placing orders, and coordinating with the supplier. As the system has the stock management, it helps to track stock levels, monitor product availability, and help to generate alerts for the low stocks. This will improve the stocks management. This system also includes the Order management which helps the administrator to create, process and track order efficiently. This administrator got the connection with the supplier. This really helps monitor for the whole order lifecycle, improve their operation efficiently and also reduce the manual movements. The third module would be the warehouse management, this module provides functionalities for tracking, picking, packing and shipping. This module helps ensure timely order fulfillment. The last module would be the revenue and expenses management. This module allows the organization to track and analyze financial aspects related to inventory. This module monitors the revenue from the sales, expenses, costs, storage expenses and shipping charges. This will help the admin to make a cost control and analyze the profit of the product.

To sum up, the combination of this module will provide an efficient solution for managing inventory, make it easy for the order between administrator and supplier, optimize stock levels, improve the warehouse operations and effectively manage revenue and expenses. This will certainly improve the customer satisfaction throughout the overall efficient order fulfillment.

2.1 Problem Statement

Business is done in mass production and there has a tendency for the management to lose track of their inventory due to increased demand in a certain business. This may lead to inefficient data records for their inventory. For instance, managers may find it difficult to keep track of the stock, purchase, order, net profit and monthly report analytics regarding their business development. This uncertainty in their inventory might be a reason for them to not operate as par with other growing business companies.

Hence, there should exist a system called inventory management system for a particular business to help them keep track of their daily production. This includes all the necessary management involving in the business such as stock management, purchase transaction management, reservation and order management and revenue and expenses management which work together to keep the business on track and in demand.

2.2 Objectives

1. To evaluate the current inventory levels and identify any areas for improvement.
2. To observe the inventory turnover rate and identify any patterns that could affect future inventory demands.
3. To offer recommendations for optimizing the inventory management system and cutting inventory expenses.
4. To create a comprehensive inventory report that can be used by management to make informed decisions about inventory levels, order fulfillment, warehouse operations, revenue and expenses.

2.3 Scope

There are in total 4 main scopes for this inventory management system. The first scope is the item and stock management. This scope contains the basic details of the system such as names, cost and description. This scope also contains stock management as you can edit the quantity of the product. The second scope is order management. This scope includes tracking the incoming and outgoing orders. The third scope is warehouse management. This scope contains movement of inventory within the warehouse, such as receiving and shipping. The last scope is reporting. This scope generates the inventory's total sales revenue and profit.

2.4 Project Timeline

Phase 1

- Information and Requirements Gathering
- Existing inventory management systems are surveyed to understand the concept of an inventory management system.
- Decide on the modules that are being implemented in this project.
- Sketch system architecture and user interfaces

Phase 2

- Figure out the implementation of OOP concepts according to modules.

Phase 3

- Coding
- Implement GUI concept into the coding system
- Combine with other modules

Phase 4

- Fix the existing bugs
- Modify the code

3.0 System Analysis and Design

3.1 Narrative Story

A group of diligent workers formerly struggled with the challenges of inventory management in a busy warehouse. Manually tracking and organizing incoming and outgoing items took many hours, which resulted in confusion, mistakes, and delayed shipments. The crew was aware that there had to be a better solution.

Here comes the Inventory Management System, a computer programme created to completely transform warehouse operations. The system brought order to the pandemonium with its simple interface and strong functions. As inventory data was immediately updated in real-time, the receiving procedure became frictionless. The team was no longer challenged by reading scrawled notes or looking for lost goods.

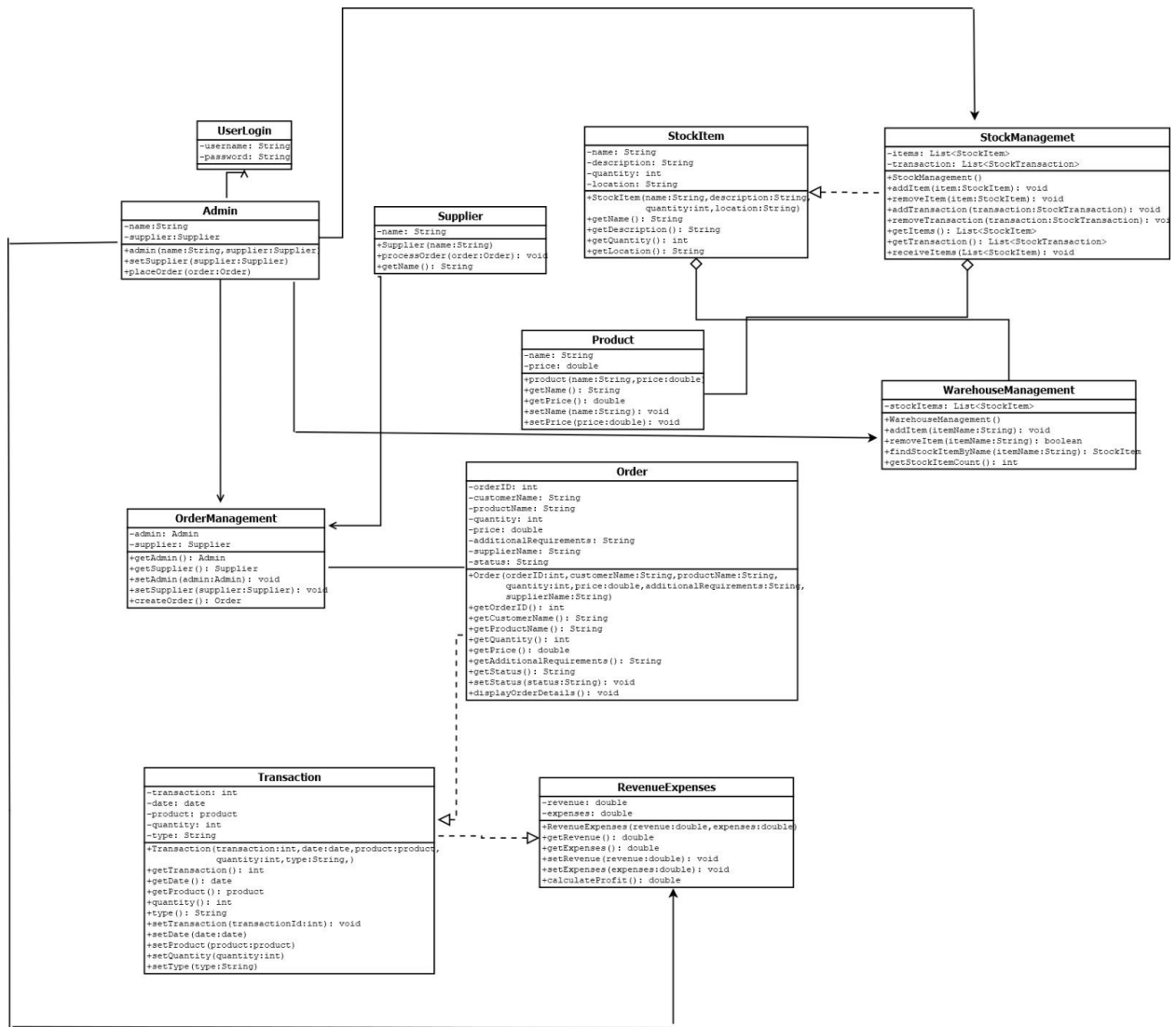
As the system intelligently identified ideal positions for each item based on their features, putaway became a breeze. The system's robust search and tracking capabilities allowed the team to find and retrieve merchandise with ease. The days of idly meandering through the warehouse were behind us.

The system's capacity to provide optimized pick lists made picking orders a much simpler operation. The group could now quickly and accurately fill customer orders, guaranteeing correctness and early delivery, with just a few clicks. No more laborious data entry or manual paperwork.

The team saw a tremendous transformation once they used the inventory management system. Stockouts and costly errors were decreased as inventory accuracy skyrocketed. Time that was before dedicated to hard labor was now used for customer service and strategic planning. The warehouse ran like a well-oiled machine, guaranteeing prompt shipments and happy clients.

The Inventory Management System ultimately emerged as the warehouse team's savior, rescuing them from the dangers of chaos and inefficiency. It unlocked a new level of productivity and success with its strong features and user-friendly layout. Finally, the team was able to concentrate on what really mattered—providing outstanding service and expanding the company.

3.2 UML Diagram - Class Diagram



4.0 Implementation

i. Assess: The first step is to assess your current inventory management practices. This includes understanding your inventory needs, current inventory levels, and reorder points. You can do this by reviewing your historical sales data, conducting inventory counts, and talking to your employees. Once you have a good understanding of your current inventory management practices, you can identify areas where you can improve.

ii. Choose: Once you have assessed your current inventory management practices, you can start to choose the right inventory management system for your business. There are many different inventory management systems available, so it's important to choose one that meets your specific needs and requirements. Some factors to consider include the size of your business, the types of products you sell, and your budget.

iii. Configure: Once you have selected an inventory management system, you need to configure it to meet your specific needs. This includes entering your product data, setting up reorder points, and configuring reports. The specific steps involved in configuring an inventory management system will vary depending on the system you choose. However, most systems will allow you to enter your product data, set up reorder points, and configure reports.

iv. Implement: Once you have configured the inventory management system, you can start to implement it. This includes training your employees on how to use the system and putting it into use. It's important to train your employees on how to use the system correctly so that they can track inventory accurately and efficiently.

v. Monitor: Once the inventory management system is in place, you need to monitor its performance. This includes checking inventory levels, reviewing reports, and investigating any discrepancies. You should also monitor the system's performance over time to make sure that it is meeting your expectations.

vi. Optimize: Over time, you may need to optimize the inventory management system to further improve your inventory accuracy, efficiency, and customer service. This may involve making changes to the system's settings or adding new features. For example, you may want to add a new report that tracks inventory turnover or make changes to the system's reorder points.

a) Define your inventory needs. This includes understanding what products or goods you need to track, how much inventory you typically have on hand, and what your reorder points are. You can do this by reviewing your historical sales data, conducting inventory counts, and talking to your employees. Once you have a good understanding of your inventory needs, you can identify areas where you can improve.

For example, you may need to track more products or goods, or you may need to increase your inventory levels. You may also need to change your reorder points so that you don't run out of stock of important items.

b) Select an inventory management system. There are many different inventory management systems available, so it's important to choose one that meets your specific needs and requirements. Some factors to consider include the size of your business, the types of products you sell, and your budget.

For example, if you have a small business with a limited budget, you may want to choose a cloud-based inventory management system. These systems are typically more affordable than on-premises systems, and they allow you to access your inventory data from anywhere.

c) Configure the inventory management system. Once you have selected an inventory management system, you need to configure it to meet your specific needs. This includes entering your product data, setting up reorder points, and configuring reports.

d) The specific steps involved in configuring an inventory management system will vary depending on the system you choose. However, most systems will allow you to enter your product data, set up reorder points, and configure reports.

For example, you may want to enter your product data by scanning barcodes or entering the information manually. You may also want to set up reorder points based on your historical sales data. And you may want to configure reports to track inventory levels, sales trends, and other important data.

5.0 Conclusion

Implementing an inventory system is a critical step for businesses of all sizes. By following the steps outlined in this project, you can ensure that your inventory management system is set up to meet your specific needs and requirements.

The first step is to define your inventory needs. This includes understanding what products or goods you need to track, how much inventory you typically have on hand, and what your reorder points are. Once you have a good understanding of your inventory needs, you can identify areas where you can improve. The next step is to select an inventory management system. There are many different inventory management systems available, so it's important to choose one that meets your specific needs and requirements. Some factors to consider include the size of your business, the types of products you sell, and your budget.

Once you have selected an inventory management system, you need to configure it to meet your specific needs. This includes entering your product data, setting up reorder points, and configuring reports. Once the system is configured, you can start to implement it. This includes training your employees on how to use the system and putting it into use.

Once the system is in place, you need to monitor it to ensure that it's working properly. This includes checking inventory levels, reviewing reports, and investigating any discrepancies. As your business changes, you may need to make adjustments to your inventory management system. This could involve adding new features, changing the way you track inventory, or even switching to a different system altogether. By following these steps, you can implement an inventory management system that will help you improve your inventory accuracy, efficiency, and customer service.

In addition to the steps outlined above, there are a few other things to keep in mind when implementing an inventory system. First, it's important to get buy-in from your employees. They need to be aware of the benefits of the new system and be willing to use it. Second, you need to make sure that the system is easy to use. If it's too complicated, your employees will be less likely to use it correctly. Finally, you need to provide ongoing training and support to your employees. This will help them to stay up-to-date on the latest changes to the system and ensure that they're using it correctly.

6.0 Reference

- Atieh, A. M., Kaylani, H., Al-abdallat, Y., Qaderi, A., Ghoul, L., Jaradat, L., & Hdairis, I. (2016). Performance improvement of inventory management system processes by an automated warehouse management system. *Procedia CIRP*, 41(1), 568–572.
<https://doi.org/10.1016/j.procir.2015.12.122>
- Berg, J. P. van den, & Zijm, W. H. M. (1999). Models for warehouse management: Classification and examples. *International Journal of Production Economics*, 59(1-3), 519–528.
[https://doi.org/10.1016/s0925-5273\(98\)00114-5](https://doi.org/10.1016/s0925-5273(98)00114-5)
- Grzegorz, M. (n.d.). VALUE-BASED INVENTORY MANAGEMENT.
https://www.ekf.vsb.cz/share/static/ekf/www.ekf.vsb.cz/export/sites/ekf/frpfi-history/.content/galerie-dokumentu/2007/dokumenty/S154_Michalski_Grzegorz.pdf
- Muller, M. (2019). Essentials of Inventory Management. In Google Books. HarperCollins Leadership.
https://books.google.com.my/books?hl=en&lr=&id=R_JWDwAAQBAJ&oi=fnd&pg=P1&dq=inventory+management&ots=7zUkJc1B8A&sig=tpPy9IS7Lap_RXSP9EKTUveaUg4&redir_esc=y#v=onepage&q=inventory%20management&f=false
- Tejesh, B. S. S., & Neeraja, S. (2018). Warehouse inventory management system using IoT and open source framework. *Alexandria Engineering Journal*, 57(4), 3817–3823.
<https://doi.org/10.1016/j.aej.2018.02.003>
- Wild, T. (2017). Best Practice in Inventory Management. In Google Books. Routledge.
https://books.google.com.my/books?hl=en&lr=&id=5jQ8DwAAQBAJ&oi=fnd&pg=PP1&dq=inventory+management&ots=tPumx6nciO&sig=OHFFEpbpxCyomEaNqXR27ZNWhH0&redir_esc=y#v=onepage&q=inventory%20management&f=false

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.util.HashMap;
import java.util.Map;
import java.util.Objects;

public class MainGUI extends JFrame {
    private final JComboBox<String> roleComboBox;
    private final JTextField usernameTextField;
    private final JPasswordField passwordField;
    private RevenueExpensesGUI revenueExpensesGUI;
    private WarehouseManagementGUI warehouseManagementGUI;
    private StockManagementGUI stockManagementGUI;
    private OrderGUI orderGUI;

    private final Map<String, Map<String, String>> registeredUsers;

    public MainGUI() {
        // Set the background color to pink pastel
        Color pinkPastel = new Color(249, 220, 215);
        getContentPane().setBackground(pinkPastel);

        setTitle("Role-Based Login");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(400, 300);
        setLayout(new FlowLayout());

        registeredUsers = new HashMap<>();
        registeredUsers.put("Admin", new HashMap<>());
        registeredUsers.put("User", new HashMap<>());

        String[] roles = {"Admin", "User"};
        roleComboBox = new JComboBox<>(roles);
        add(roleComboBox);

        JLabel usernameLabel = new JLabel("Username:");
        add(usernameLabel);

        usernameTextField = new JTextField(20);
        add(usernameTextField);
    }

```

```

JLabel passwordLabel = new JLabel("Password:");
add(passwordLabel);

passwordField = new JPasswordField(20);
add(passwordField);

JButton loginButton = new JButton("Login");
loginButton.addActionListener(e -> {
    String role = (String) roleComboBox.getSelectedItem();
    String username = usernameTextField.getText();
    String password = new String(passwordField.getPassword());

    try {
        if (validateCredentials(role, username, password)) {
            JOptionPane.showMessageDialog(null, role + " login successful!");
            assert role != null;
            showMainMenu(role); // Show the main menu based on the user's role
        } else {
            JOptionPane.showMessageDialog(null, "Invalid credentials!");
        }
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(null, "An error occurred: " + ex.getMessage());
    }
});
add(loginButton);

JButton registerButton = new JButton("Register");
registerButton.addActionListener(e -> openRegisterPanel());
add(registerButton);

setVisible(true);
}

private boolean validateCredentials(String role, String username, String password) {
    if (registeredUsers.containsKey(role)) {
        Map<String, String> users = registeredUsers.get(role);
        if (users.containsKey(username)) {
            String storedPassword = users.get(username);
            return storedPassword.equals(password);
        }
    }
}

```

```

    return false;
}

private void showMainMenu(String role) {
    JFrame mainMenuFrame = new JFrame(role + " Main Menu");
    mainMenuFrame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    mainMenuFrame.setSize(400, 300);
    mainMenuFrame.setLayout(new FlowLayout());

    // Set the background color to pink pastel
    Color pinkPastel = new Color(249, 220, 215);
    mainMenuFrame.getContentPane().setBackground(pinkPastel);

    if (role.equals("Admin")) {
        JButton manageRevenueExpensesButton = new JButton("Manage Revenue and
Expenses");
        mainMenuFrame.add(manageRevenueExpensesButton);
        // Add action listener for the manage revenue and expenses button
        manageRevenueExpensesButton.addActionListener(e -> openRevenueExpensesGUI());
        JButton manageWarehouseManagementButton = new JButton("Manage Warehouse");
        mainMenuFrame.add(manageWarehouseManagementButton);
        manageWarehouseManagementButton.addActionListener(e ->
openWarehouseManagementGUI());
        JButton manageStockManagementButton = new JButton("Manage Stock");
        mainMenuFrame.add(manageStockManagementButton);
        manageStockManagementButton.addActionListener(e -> openStockManagementGUI());
        JButton placeOrderButton = new JButton("Place Order");
        mainMenuFrame.add(placeOrderButton);
        // Add action listener for the place order button
        placeOrderButton.addActionListener(e -> {
            mainMenuFrame.dispose(); // Close the main menu window
            showOrderGUI();
        });
    } else if (role.equals("User")) {
        JButton manageStockManagementButton = new JButton("Manage Stock");
        mainMenuFrame.add(manageStockManagementButton);
        manageStockManagementButton.addActionListener(e -> openStockManagementGUI());
        JButton manageWarehouseManagementButton = new JButton("Manage Warehouse");
        mainMenuFrame.add(manageWarehouseManagementButton);
        manageWarehouseManagementButton.addActionListener(e ->
openWarehouseManagementGUI());
    }
}

```

```

        JButton exitButton = new JButton("Exit");
        mainMenuFrame.add(exitButton);
        exitButton.addActionListener(e -> {
            mainMenuFrame.dispose();
            System.exit(0);
        });

        mainMenuFrame.setVisible(true);
    }

    private void openRevenueExpensesGUI() {
        revenueExpensesGUI = new RevenueExpensesGUI();
        revenueExpensesGUI.setVisible(true);
        revenueExpensesGUI.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        revenueExpensesGUI.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosed(WindowEvent e) {
                revenueExpensesGUI.dispose();
                showMainMenu((String) Objects.requireNonNull(roleComboBox.getSelectedItem()));
            }
        });
    }

    private void openWarehouseManagementGUI() {
        warehouseManagementGUI = new WarehouseManagementGUI();
        warehouseManagementGUI.setVisible(true);
        warehouseManagementGUI.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        warehouseManagementGUI.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosed(WindowEvent e) {
                warehouseManagementGUI.dispose();
                showMainMenu((String) Objects.requireNonNull(roleComboBox.getSelectedItem()));
            }
        });
    }

    private void openStockManagementGUI() {
        stockManagementGUI = new StockManagementGUI();
        stockManagementGUI.setVisible(true);
        stockManagementGUI.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        stockManagementGUI.addWindowListener(new WindowAdapter() {

```



```

        @Override
        public void windowClosed(WindowEvent e) {
            stockManagementGUI.dispose();
            showMainMenu((String) Objects.requireNonNull(roleComboBox.getSelectedItem()));
        }
    });
}

```

```

private void showOrderGUI() {
    orderGUI = new OrderGUI();
    orderGUI.setVisible(true);
    orderGUI.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    orderGUI.addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosed(WindowEvent e) {
            try {
                orderGUI.dispose();
            } catch (InterruptedException ex) {
                throw new RuntimeException(ex);
            }
            showMainMenu((String) Objects.requireNonNull(roleComboBox.getSelectedItem()));
        }
    });
}

```

```

private void openRegisterPanel() {
    JFrame registerFrame = new JFrame("Register New User");
    registerFrame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    registerFrame.setSize(400, 300);
    registerFrame.setLayout(new FlowLayout());

    // Set the background color to pink pastel
    Color pinkPastel = new Color(249, 220, 215);
    registerFrame.getContentPane().setBackground(pinkPastel);

    JLabel userIdLabel = new JLabel("User ID:");
    JTextField userIdTextField = new JTextField(20);
    registerFrame.add(userIdLabel);
    registerFrame.add(userIdTextField);

    JLabel userTypeLabel = new JLabel("User Type:");
}

```

```

        JComboBox<String> userTypeComboBox = new JComboBox<>(new String[]{"Admin",
"User"});
        registerFrame.add(userTypeLabel);
        registerFrame.add(userTypeComboBox);

        JLabel passwordLabel = new JLabel("Password:");
        JPasswordField passwordField = new JPasswordField(20);
        registerFrame.add(passwordLabel);
        registerFrame.add(passwordField);

        JButton registerButton = new JButton("Register");
        registerButton.addActionListener(e -> {
            String userId = userIdTextField.getText();
            String userType = (String) userTypeComboBox.getSelectedItem();
            String password = new String(passwordField.getPassword());

            if (registeredUsers.containsKey(userType)) {
                Map<String, String> users = registeredUsers.get(userType);
                if (!users.containsKey(userId)) {
                    users.put(userId, password);
                    JOptionPane.showMessageDialog(null, "User registered successfully!");
                    registerFrame.dispose();
                } else {
                    JOptionPane.showMessageDialog(null, "User already exists!");
                }
            } else {
                JOptionPane.showMessageDialog(null, "Invalid user type!");
            }
        });
        registerFrame.add(registerButton);

        registerFrame.setVisible(true);
    }

    public static void main(String[] args) {
        new MainGUI();
    }
}

```

```
public class Main {
    public static void main(String[] args) {
        // Create the admin object
        Admin admin = new Admin("Admin Name");

        // Create the order management object and set the admin
        OrderManagement orderManagement = new OrderManagement();
        orderManagement.setAdmin(admin);

        // Create the supplier object with a name
        Supplier supplier = new Supplier("Supplier Name");
        orderManagement.setSupplier(supplier);

        // Create the OrderGUI instance to show the GUI for capturing user input
        Order order;
        OrderGUI orderGUI = new OrderGUI();

        // Retrieve the order object from the GUI
        order = orderGUI.getOrder();

        if (order != null) {
            // Display order information
            order.displayOrderDetails();
        }
    }
}
```

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
```

```
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.text.DecimalFormat;
import java.util.Scanner;

public class OrderGUI {
    private final JFrame frame;
    private Order order;

    public OrderGUI() {
        // Create the main frame
        frame = new JFrame("Order Details");
        frame.setSize(400, 400);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(new BorderLayout());
        frame.setVisible(true);

        // Set the background color to pink pastel
        Color pinkPastel = new Color(249, 220, 215);
        frame.getContentPane().setBackground(pinkPastel);

        // Create the panel to hold order details
        JPanel orderPanel = new JPanel();
        orderPanel.setLayout(new GridLayout(7, 2, 10, 10));
        orderPanel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));

        // Add order details components to the panel
        JLabel orderIdLabel = new JLabel("Order ID:");
        JTextField orderIdField = new JTextField();
        orderPanel.add(orderIdLabel);
        orderPanel.add(orderIdField);

        JLabel customerNameLabel = new JLabel("Customer Name:");
        JTextField customerNameField = new JTextField();
        orderPanel.add(customerNameLabel);
        orderPanel.add(customerNameField);

        JLabel productNameLabel = new JLabel("Product Name:");
        JTextField productNameField = new JTextField();
        orderPanel.add(productNameLabel);
        orderPanel.add(productNameField);
    }
}
```

```
JLabel quantityLabel = new JLabel("Quantity:");
JTextField quantityField = new JTextField();
orderPanel.add(quantityLabel);
orderPanel.add(quantityField);
```

```
JLabel priceLabel = new JLabel("Price:");
JTextField priceField = new JTextField();
orderPanel.add(priceLabel);
orderPanel.add(priceField);
```

```
JLabel additionalRequirementsLabel = new JLabel("Additional Requirements:");
JTextField additionalRequirementsField = new JTextField();
orderPanel.add(additionalRequirementsLabel);
orderPanel.add(additionalRequirementsField);
```

```
JLabel supplierNameLabel = new JLabel("Supplier Name:");
JTextField supplierNameField = new JTextField();
orderPanel.add(supplierNameLabel);
orderPanel.add(supplierNameField);
```

```
// Create the place order button
JButton placeOrderButton = new JButton("Place Order");
placeOrderButton.setFont(new Font("Arial", Font.BOLD, 14));
// Set the background color to pink
pinkPastel = new Color(249, 220, 215);
placeOrderButton.setBackground(pinkPastel);
```

```
placeOrderButton.setForeground(Color.WHITE);
```

```
placeOrderButton.addActionListener(e -> {
    // Create a new Order object
    int orderId = Integer.parseInt(orderIdField.getText());
    String customerName = customerNameField.getText();
    String productName = productNameField.getText();
    int quantity = Integer.parseInt(quantityField.getText());
    double price = Double.parseDouble(priceField.getText());
    String additionalRequirements = additionalRequirementsField.getText();
    String supplierName = supplierNameField.getText();

    // Assign the values to the Order object
```

```

        OrderGUI.this.order = new Order(orderId, customerName, productName, quantity,
price, additionalRequirements, supplierName);

        // Create the OrderSuccessGUI instance to show the success message and order details
        OrderSuccessGUI successGUI = new OrderSuccessGUI(OrderGUI.this.order);

        // Close the GUI
        frame.dispose();
    });

    // Create a panel for the button
    JPanel buttonPanel = new JPanel();
    buttonPanel.setLayout(new FlowLayout(FlowLayout.CENTER));
    buttonPanel.add(placeOrderButton);

    // Create a panel for the content
    JPanel contentPanel = new JPanel();
    contentPanel.setLayout(new BorderLayout());
    contentPanel.add(orderPanel, BorderLayout.CENTER);
    contentPanel.add(buttonPanel, BorderLayout.SOUTH);

    // Add the content panel to the frame
    frame.add(contentPanel);

    // Set the frame visible
    frame.setVisible(true);
}

public Order getOrder() {
    return order;
}

public void addWindowListener(WindowAdapter windowAdapter) {
    frame.addWindowListener(windowAdapter);
}

public void setOrderListener(OrderGUI.OrderListener orderListener) {
}

public void setVisible(boolean b) {
    if (b) {
        // Set the visibility of the current frame to true
    }
}

```

```

        super.notify();
    } else {
        // Set the visibility of the current frame to false
        super.notify();
    }
}

public void setDefaultCloseOperation(int disposeOnClose) {
    switch (disposeOnClose) {
        case JFrame.HIDE_ON_CLOSE:
            setDefaultCloseOperation(JFrame.HIDE_ON_CLOSE);
            break;
        case JFrame.DISPOSE_ON_CLOSE:
            setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
            break;
        case JFrame.EXIT_ON_CLOSE:
            setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            break;
        default:
            addWindowListener(new WindowAdapter() {
                @Override
                public void windowClosing(WindowEvent e) {}
            });
    }
}

public void dispose() throws InterruptedException {
    super.wait();
}

public static class OrderListener {
    void orderPlaced(Order order) {

    }
}

class OrderSuccessGUI {
    private final JFrame frame;
    private Order order;

    public OrderSuccessGUI(Order order) {

```

```
// Create the main frame
frame = new JFrame("Order Success");
frame.setSize(400, 300);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setLayout(new BorderLayout());

// Create the panel to hold order details
JPanel orderPanel = new JPanel();
orderPanel.setLayout(new GridLayout(8, 2));
frame.getContentPane().setBackground(Color.PINK); // Set the background color to pink

// Add order details components to the panel
JLabel orderIdLabel = new JLabel("Order ID:");
JLabel orderIdValue = new JLabel(String.valueOf(order.getId()));
orderPanel.add(orderIdLabel);
orderPanel.add(orderIdValue);

JLabel customerNameLabel = new JLabel("Customer Name:");
JLabel customerNameValue = new JLabel(order.getCustomerName());
orderPanel.add(customerNameLabel);
orderPanel.add(customerNameValue);

JLabel productNameLabel = new JLabel("Product Name:");
JLabel productNameValue = new JLabel(order.getProductName());
orderPanel.add(productNameLabel);
orderPanel.add(productNameValue);

JLabel quantityLabel = new JLabel("Quantity:");
JLabel quantityValue = new JLabel(String.valueOf(order.getQuantity()));
orderPanel.add(quantityLabel);
orderPanel.add(quantityValue);

JLabel priceLabel = new JLabel("Price:");
JLabel priceValue = new JLabel(formatCurrency(order.getPrice()));
orderPanel.add(priceLabel);
orderPanel.add(priceValue);

JLabel additionalRequirementsLabel = new JLabel("Additional Requirements:");
JLabel additionalRequirementsValue = new JLabel(order.getAdditionalRequirements());
orderPanel.add(additionalRequirementsLabel);
orderPanel.add(additionalRequirementsValue);
```



```

JLabel supplierNameLabel = new JLabel("Supplier Name:");
JLabel supplierNameValue = new JLabel(order.getSupplierName());
orderPanel.add(supplierNameLabel);
orderPanel.add(supplierNameValue);

JLabel statusLabel = new JLabel("Status:");
JLabel statusValue = new JLabel(order.getStatus());
orderPanel.add(statusLabel);
orderPanel.add(statusValue);

// Create the remove order button
JButton removeOrderButton = new JButton("OK");
removeOrderButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        int orderId = order.getId();
        // Here you should call the removeOrder method from OrderManagement
        // For now, we will display a message that the order is removed
        JOptionPane.showMessageDialog(frame, orderId + ": SUCCESSFULLY ORDERED");
        frame.dispose();
    }
});

// Create a panel to hold the remove button
JPanel buttonPanel = new JPanel();
buttonPanel.add(removeOrderButton);

// Add the order panel and button panel to the frame
frame.add(orderPanel, BorderLayout.CENTER);
frame.add(buttonPanel, BorderLayout.SOUTH);

// Set the frame visible
frame.setVisible(true);
}

// Helper method to format the currency value
private String formatCurrency(double value) {
    DecimalFormat decimalFormat = new DecimalFormat("RM #,##0.00");
    return decimalFormat.format(value);
}

public void dispose() {
    frame.dispose();
}

```

```

    }

    public void setOrder(Order order) {
        this.order = order;
    }

    public void addWindowListener(WindowAdapter windowAdapter) {
        frame.addWindowListener(windowAdapter);
    }

    public void setVisible(boolean b) {
        if (b) {
            // Set the visibility of the main menu frame
            setVisible(true);
        } else {
            // Set the visibility of the main menu frame to false
            setVisible(false);
        }
    }
}

class OrderManagement {
    private Admin admin;
    private Supplier supplier;

    public void setAdmin(Admin admin) {
        this.admin = admin;
    }

    public Admin getAdmin() {
        return admin;
    }

    public void setSupplier(Supplier supplier) {
        this.supplier = supplier;
    }

    public Supplier getSupplier() {
        return supplier;
    }

    public Order createOrder() {

```

```

// Create a Scanner object for user input
Scanner scanner = new Scanner(System.in);

// Get user inputs for order details
System.out.print("Enter Order ID: ");
int orderId = scanner.nextInt();
scanner.nextLine(); // Consume the remaining newline character

System.out.print("Enter Customer Name: ");
String customerName = scanner.nextLine();

System.out.print("Enter Product Name: ");
String productName = scanner.nextLine();

System.out.print("Enter Quantity: ");
int quantity = scanner.nextInt();

System.out.print("Enter Price: ");
double price = scanner.nextDouble();
scanner.nextLine(); // Consume the remaining newline character

System.out.print("Enter Additional Requirements: ");
String additionalRequirements = scanner.nextLine();

// Create a new Order object
Order order = new Order(orderId, customerName, productName, quantity,
    price, additionalRequirements, supplier.getName());

return order;
}
}
class Order {
    private final int orderId;
    private final String customerName;
    private final String productName;
    private final int quantity;
    private final double price;
    private final String additionalRequirements;
    private String supplierName;
    private String status;

```

```
public Order(int orderId, String customerName, String productName, int quantity, double
price, String additionalRequirements, String supplierName) {
    this.orderId = orderId;
    this.customerName = customerName;
    this.productName = productName;
    this.quantity = quantity;
    this.price = price;
    this.additionalRequirements = additionalRequirements;
    this.supplierName = supplierName;
    this.status = "Pending"; // Initial status is set to 'Pending'
}

public int getOrderId() {
    return orderId;
}

public String getCustomerName() {
    return customerName;
}

public String getProductName() {
    return productName;
}

public int getQuantity() {
    return quantity;
}

public double getPrice() {
    return price;
}

public String getAdditionalRequirements() {
    return additionalRequirements;
}

public String getSupplierName() {
    return supplierName;
}

public String getStatus() {
    return status;
}
```

```

    }

    public void setStatus(String status) {
        this.status = status;
    }

    public void displayOrderDetails() {
        System.out.println("Order Details:");
        System.out.println("Order ID: " + orderId);
        System.out.println("Customer Name: " + customerName);
        System.out.println("Product Name: " + productName);
        System.out.println("Quantity: " + quantity);
        System.out.println("Price: RM " + price);
        System.out.println("Additional Requirements: " + additionalRequirements);
        System.out.println("Supplier Name: " + supplierName);
        System.out.println("Status: " + status);
    }

    public void setSupplierName() {
        this.supplierName=supplierName;
    }
}

class Admin {
    private final String name;

    public Admin(String name) {
        this.name = name;
    }

    public void placeOrder(Order order, Supplier supplier) {
        // Process the order using the supplier
        supplier.processOrder(order);

        // Create the OrderSuccessGUI instance to display the order success message
        OrderGUI successGUI = new OrderGUI();
    }
}

class Supplier {
    private final String name;

    public Supplier(String name) {
        this.name = name;
    }
}

```

```

    }

    public void processOrder(Order order) {
        // Update the order status
        order.setStatus("Processing");

        // Perform any necessary actions to process the order
        // For example, sending the order to the supplier's system

        // Display a message to the user
        System.out.println("Order processed by supplier: " + name);
    }

    public String getName() {
        return name;
    }
}

```

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

public class RevenueExpensesGUI extends JFrame {
    private final List<Transaction> transactions;
    private RevenueManager revenueManager;
    private ExpensesManager expensesManager;
    private final JLabel revenueLabel;
    private final JLabel expensesLabel;

    public RevenueExpensesGUI() {
        transactions = new ArrayList<>();
    }
}

```

```
setTitle("Revenue and Expenses Management");
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setSize(500, 500);
setLayout(new FlowLayout());
getContentPane().setBackground(Color.PINK); // Set the background color to pink
```

```
JButton addTransactionButton = new JButton("Add Transaction");
addTransactionButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        TransactionDialog dialog = new TransactionDialog(RevenueExpensesGUI.this);
        dialog.setVisible(true);
    }
});
```

```
JButton calculateButton = new JButton("Calculate");
calculateButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        double revenue = revenueManager.calculateRevenue();
        double expenses = expensesManager.calculateExpenses();
        double netProfit = revenue - expenses;

        revenueLabel.setText("Total Revenue: RM" + revenue);
        expensesLabel.setText("Total Expenses: RM" + expenses);
        JOptionPane.showMessageDialog(RevenueExpensesGUI.this,
            "Net Profit: RM" + netProfit,
            "Net Profit",
            JOptionPane.INFORMATION_MESSAGE);
    }
});
```

```
revenueLabel = new JLabel("Total Revenue: ");
expensesLabel = new JLabel("Total Expenses: ");
```

```
add(addTransactionButton);
add(calculateButton);
add(revenueLabel);
add(expensesLabel);
}
```

```
public void addTransaction(Transaction transaction) {
```

```

        transactions.add(transaction);
        revenueManager = new RevenueManager(transactions);
        expensesManager = new ExpensesManager(transactions);
    }
}

```

```

class Transaction {
    private final int transactionId;
    private final Date date;
    private final Product product;
    private final int quantity;
    private final String type; // "sale" or "purchase"

    public Transaction(int transactionId, Date date, Product product, int quantity, String type) {
        this.transactionId = transactionId;
        this.date = date;
        this.product = product;
        this.quantity = quantity;
        this.type = type;
    }

    // Getters and setters

    public Product getProduct() {
        return product;
    }

    public int getQuantity() {
        return quantity;
    }

    public String getType() {
        return type;
    }
}

```

```

class TransactionDialog extends JDialog {
    private final JTextField transactionIdField;
    private final JTextField dateField;
    private final JTextField productNameField;
    private final JTextField priceField;
}

```



```

private final JTextField quantityField;
private final JComboBox<String> typeComboBox;

public TransactionDialog(JFrame parent) {
    super(parent, "Add Transaction", true);
    setSize(500, 500);
    setLayout(new GridLayout(7, 2));

    JLabel transactionIdLabel = new JLabel("Transaction ID:");
    transactionIdField = new JTextField();
    JLabel dateLabel = new JLabel("Date:");
    dateField = new JTextField();
    JLabel productNameLabel = new JLabel("Product Name:");
    productNameField = new JTextField();
    JLabel priceLabel = new JLabel("Price:");
    priceField = new JTextField();
    JLabel quantityLabel = new JLabel("Quantity:");
    quantityField = new JTextField();
    JLabel typeLabel = new JLabel("Type:");
    typeComboBox = new JComboBox<>(new String[]{"Sale", "Purchase"});

    JButton addButton = new JButton("Add");
    addButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            int transactionId = Integer.parseInt(transactionIdField.getText());
            Date date = new Date(dateField.getText());
            String productName = productNameField.getText();
            double price = Double.parseDouble(priceField.getText());
            int quantity = Integer.parseInt(quantityField.getText());
            String type = typeComboBox.getSelectedItem().toString();

            Product product = new Product(productName, price);
            Transaction transaction = new Transaction(transactionId, date, product, quantity,
type);

            RevenueExpensesGUI parent = (RevenueExpensesGUI) getParent();
            parent.addTransaction(transaction);

            dispose();
        }
    });
}

```

```
        add(transactionIdLabel);
        add(transactionIdField);
        add(dateLabel);
        add(dateField);
        add(productNameLabel);
        add(productNameField);
        add(priceLabel);
        add(priceField);
        add(quantityLabel);
        add(quantityField);
        add(typeLabel);
        add(typeComboBox);
        add(addButton);
    }
}
```

```
class Product {
    private String name;
    private double price;

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }

    // Getters and setters
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }
}
```

```
}
```

```
-----  
  
import java.util.List;
```

```
public class RevenueManager {  
    private List<Transaction> transactions;  
  
    public RevenueManager(List<Transaction> transactions) {  
        this.transactions = transactions;  
    }  
  
    public double calculateRevenue() {  
        double revenue = 0;  
        for (Transaction transaction : transactions) {  
            if ("Sale".equalsIgnoreCase(transaction.getType())) {  
                revenue += transaction.getProduct().getPrice() * transaction.getQuantity();  
            }  
        }  
        return revenue;  
    }  
}  
  
class ExpensesManager {  
    private List<Transaction> transactions;  
  
    public ExpensesManager(List<Transaction> transactions) {  
        this.transactions = transactions;  
    }  
  
    public double calculateExpenses() {  
        double expenses = 0;  
        for (Transaction transaction : transactions) {  
            if ("Purchase".equalsIgnoreCase(transaction.getType())) {  
                expenses += transaction.getProduct().getPrice() * transaction.getQuantity();  
            }  
        }  
    }  
}
```

```

    }
    }
    return expenses;
}
}

```

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.HashMap;
import java.util.Map;

```

```

public class RoleBasedLoginGUI extends JFrame {

    //creates a combo box for the roles which are admin and user
    private final JComboBox<String> roleComboBox;
    //creates a label for username
    private final JLabel usernameLabel;
    //creates a label for password
    private final JLabel passwordLabel;
    //creates a text field for users to type into for username
    private final JTextField usernameTextField;
    //creates a text field for users to input their password
    private final JPasswordField passwordField;
    //creates a login button
    private final JButton loginButton;
    //creates a register button
    private final JButton registerButton;

    private final Map<String, Map<String, String>> registeredUsers;

    public RoleBasedLoginGUI () {
        //sets frame requirements
        setTitle("Role-Based Login");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

```
setSize(400, 300);
setLayout(new FlowLayout());
Color color=new Color(204, 204, 255);
getContentPane().setBackground(color);
getContentPane().setBackground(Color.PINK); // Set the background color to pink
```

```
//creates separate hashmap for the admin and user info to be stored into
registeredUsers = new HashMap<>();
registeredUsers.put("Admin", new HashMap<>());
registeredUsers.put("User", new HashMap<>());
```

```
//creates the role combo box
String[] roles = {"Admin", "User"};
roleComboBox = new JComboBox<>(roles);
add(roleComboBox);
```

```
usernameLabel = new JLabel("Username:");
//sets the layout of the username label
add(usernameLabel,FlowLayout.LEFT);
```

```
usernameTextField = new JTextField(20);
//sets the layout for the username text field
add(usernameTextField,FlowLayout.CENTER);
```

```
passwordLabel = new JLabel("Password:");
add(passwordLabel);
```

```
passwordField = new JPasswordField(20);
add(passwordField);
```

```
loginButton = new JButton("Login");
loginButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        //gets the input for role,username and password
        String role = (String) roleComboBox.getSelectedItem();
        String username = usernameTextField.getText();
        String password = new String(passwordField.getPassword());

        try {
```

```

        //checks if the input is valid or not
        if (validateCredentials(role, username, password)) {
            JOptionPane.showMessageDialog(null, role + " login successful!");
            showMainMenu(role); // Show the main menu based on the user's role
            dispose(); // Close the login frame after successful login
        } else {
            JOptionPane.showMessageDialog(null, "Invalid credentials!");
        }
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(null, "An error occurred: " + ex.getMessage());
    }
}

});
add(loginButton, new BorderLayout());

//register button for new users
registerButton = new JButton("Register");
registerButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        //opens a new panel where new users can register themselves with
        username,password and user type
        openRegisterPanel();
    }
});
add(registerButton);

setVisible(true);
}

//checks if there is already information stored in the hashmap which indicates registered
users
private boolean validateCredentials(String role, String username, String password) {
    if (registeredUsers.containsKey(role)) {
        Map<String, String> users = registeredUsers.get(role);
        if (users.containsKey(username)) {
            String storedPassword = users.get(username);
            return storedPassword.equals(password);
        }
    }
}
return false;
}

```

```

private void showMainMenu(String role) {
    JFrame mainMenuFrame = new JFrame(role + " Main Menu");
    mainMenuFrame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    mainMenuFrame.setSize(400, 300);
    mainMenuFrame.setLayout(new FlowLayout());
    JDialog registerFrame = null;
    registerFrame.getContentPane().setBackground(Color.PINK);

    if (role.equals("Admin")) {
        JButton manageRevenueExpensesButton = new JButton("Manage Revenue and
Expenses");
        mainMenuFrame.add(manageRevenueExpensesButton);
        // Add action listener for the manage revenue and expenses button
        manageRevenueExpensesButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // Open the revenue and expenses management GUI here
                JOptionPane.showMessageDialog(null, "Open Revenue and Expenses
Management GUI");
            }
        });
    } else if (role.equals("User")) {
        JButton manageStockButton = new JButton("Manage Stock");
        mainMenuFrame.add(manageStockButton);
        // Add action listener for the manage stock button
        manageStockButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // Open the stock management GUI here
                JOptionPane.showMessageDialog(null, "Open Stock Management GUI");
            }
        });
    }

    mainMenuFrame.setVisible(true);
}

private void openRegisterPanel() {
    //the basic requirements for the frame
    JFrame registerFrame = new JFrame("Register New User");

```

```

registerFrame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
registerFrame.setSize(400, 300);
registerFrame.setLayout(new FlowLayout());
registerFrame.getContentPane().setBackground(Color.PINK);

JLabel userIdLabel = new JLabel("User ID:");
JTextField userIdTextField = new JTextField(20);
//sets the layout of the userid label
registerFrame.add(userIdLabel,FlowLayout.LEFT);
registerFrame.add(userIdTextField);

JLabel userTypeLabel = new JLabel("User Type:");
JComboBox<String> userTypeComboBox = new JComboBox<>(new String[]{"Admin",
"User"});
//sets the layout of the user type label
registerFrame.add(userTypeLabel,FlowLayout.RIGHT);
registerFrame.add(userTypeComboBox);

JLabel passwordLabel = new JLabel("Password:");
JPasswordField passwordField = new JPasswordField(20);
registerFrame.add(passwordLabel);
registerFrame.add(passwordField);

JButton registerButton = new JButton("Register");
registerButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        String userId = userIdTextField.getText();
        String userType = (String) userTypeComboBox.getSelectedItem();
        String password = new String(passwordField.getPassword());

        //checks if the information already exists in the hashmap,if not stores and creates a
new user
        if (registeredUsers.containsKey(userType)) {
            Map<String, String> users = registeredUsers.get(userType);
            if (!users.containsKey(userId)) {
                users.put(userId, password);
                JOptionPane.showMessageDialog(null, "User registered successfully!");
                registerFrame.dispose();
            } else {
                JOptionPane.showMessageDialog(null, "User already exists!");
            }
        }
    }
});

```



```

        }
    } else {
        JOptionPane.showMessageDialog(null, "Invalid user type!");
    }
}
});
registerFrame.add(registerButton);

registerFrame.setVisible(true);
}

//the login GUI is initialized and displayed on the screen.
public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
            new RoleBasedLoginGUI ();
        }
    });
}
}
}

```

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class StockManagementGUI extends JFrame {
    private final DefaultListModel<StockItem> itemsListModel;
    private final JList<StockItem> itemsList;

    public StockManagementGUI() {
        // Set up the frame
        setTitle("Stock Management");
    }
}

```

```

setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setSize(400, 300);
setLocationRelativeTo(null);
getContentPane().setBackground(Color.PINK); // Set the background color to pink

// Create the main panel
JPanel mainPanel = new JPanel();
mainPanel.setLayout(new BorderLayout());
getContentPane().add(mainPanel);
mainPanel.setBackground(Color.PINK); // Set the background color to pink
getContentPane().add(mainPanel);

// Create the items list
itemsListModel = new DefaultListModel<>();
itemsList = new JList<>(itemsListModel);
JScrollPane itemsScrollPane = new JScrollPane(itemsList);
mainPanel.add(itemsScrollPane, BorderLayout.CENTER);

// Create buttons
JButton addItemButton = new JButton("Add Item");
JButton removeItemButton = new JButton("Remove Item");
JButton displayDetailsButton = new JButton("Display Details");
JPanel buttonPanel = new JPanel();
buttonPanel.add(addItemButton);
buttonPanel.add(removeItemButton);
buttonPanel.add(displayDetailsButton);
mainPanel.add(buttonPanel, BorderLayout.SOUTH);

// Add action listeners directly to the buttons
addItemButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        addItem();
    }
});

removeItemButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        removeItem();
    }
});

```

```

displayDetailsButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        displayDetails();
    }
});

// Initialize the GUI with some sample data
initializeSampleData();
}

private void initializeSampleData() {
    StockItem item1 = new StockItem("Item 1", "Description 1", 10, "Location 1");
    StockItem item2 = new StockItem("Item 2", "Description 2", 5, "Location 2");
    StockItem item3 = new StockItem("Item 3", "Description 3", 8, "Location 3");

    addItemToList(item1);
    addItemToList(item2);
    addItemToList(item3);
}

private void addItemToList(StockItem item) {
    itemListModel.addElement(item);
}

private void removeItemFromList(StockItem item) {
    itemListModel.removeElement(item);
}

private void addItem() {
    // Show input dialogs to get item details from the user
    String name = JOptionPane.showInputDialog("Enter item name:");
    String description = JOptionPane.showInputDialog("Enter item description:");
    String quantityStr = JOptionPane.showInputDialog("Enter item quantity:");
    int quantity = Integer.parseInt(quantityStr);
    String location = JOptionPane.showInputDialog("Enter item location:");

    // Create a new StockItem object with the provided details
    StockItem newItem = new StockItem(name, description, quantity, location);

```

```

        // Add the item to the list
        addItemToList(newItem);
    }

    private void removeItem() {
        // Get the selected item from the list
        StockItem selectedItem = itemsList.getSelectedValue();

        // Remove the selected item from the list
        if (selectedItem != null) {
            removeItemFromList(selectedItem);
        } else {
            JOptionPane.showMessageDialog(this, "No item selected.");
        }
    }

    private void displayDetails() {
        // Get the selected item from the list
        StockItem selectedItem = itemsList.getSelectedValue();

        // Display the details in a dialog box
        if (selectedItem != null) {
            String details = "Item: " + selectedItem.getName() + "\n"
                + "Description: " + selectedItem.getDescription() + "\n"
                + "Quantity: " + selectedItem.getQuantity() + "\n"
                + "Location: " + selectedItem.getLocation();

            JOptionPane.showMessageDialog(this, details, "Item Details",
JOptionPane.INFORMATION_MESSAGE);
        } else {
            JOptionPane.showMessageDialog(this, "No item selected.", "Error",
JOptionPane.ERROR_MESSAGE);
        }
    }
}

class StockTransaction {
    public StockTransaction(String orderId, StockItem item, int quantity, TransactionType
transactionType) {
        this.item = item;
        this.transactionType = transactionType;
    }
}

```

```

public enum TransactionType {
    ADD, // Represents adding stock
    REMOVE // Represents removing stock
}

private TransactionType transactionType;
private StockItem item;
private int quantity;

public StockTransaction(String type, StockItem item, int quantity) {
    this.type = type;
    this.item = item;
    this.quantity = quantity;
    this.transactionType = transactionType;
}

private String type;
}

class StockItem {
    private final String name;
    private String description;
    private int quantity;
    private String location;

    public StockItem(String itemName) {
        this.name = itemName;
    }

    public StockItem(String name, String description, int quantity, String location) {
        this.name = name;
        this.description = description;
        this.quantity = quantity;
        this.location = location;
    }

    public StockItem(String name, int quantity) {
        this.name = name;
        this.quantity = quantity;
    }
}

```

```
public String getName() {  
    return name;  
}  
  
public String getDescription() {  
    return description;  
}  
  
public int getQuantity() {  
    return quantity;  
}  
  
public String getLocation() {  
    return location;  
}  
}
```

```
import javax.swing.*;  
import java.awt.*;  
import java.awt.event.ActionEvent;  
import java.awt.event.ActionListener;  
import java.util.ArrayList;  
import java.util.List;  
  
public class WarehouseManagementGUI extends JFrame {  
    private DefaultListModel<StockItem> itemsListModel;  
    private JList<StockItem> itemsList;  
    private int itemCount;  
    private JLabel itemCountLabel;  
  
    public WarehouseManagementGUI() {  
        // Set up the frame  
        setTitle("Warehouse Management");  
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    }  
}
```

```
setSize(400, 300);
setLocationRelativeTo(null);
getContentPane().setBackground(Color.PINK); // Set the background color to pink
```

```
// Create the main panel
JPanel mainPanel = new JPanel();
mainPanel.setLayout(new BorderLayout());
getContentPane().add(mainPanel);
mainPanel.setBackground(Color.PINK); // Set the background color to pink
getContentPane().add(mainPanel);
```

```
// Create the items list
itemsListModel = new DefaultListModel<>();
itemsList = new JList<>(itemsListModel);
JScrollPane itemsScrollPane = new JScrollPane(itemsList);
mainPanel.add(itemsScrollPane, BorderLayout.CENTER);
```

```
// Create buttons
JButton addItemButton = new JButton("Add Item");
JButton removeItemButton = new JButton("Remove Item");
JButton displayDetailsButton = new JButton("Display Details");
```

```
JPanel buttonPanel = new JPanel();
buttonPanel.add(addItemButton);
buttonPanel.add(removeItemButton);
buttonPanel.add(displayDetailsButton);
mainPanel.add(buttonPanel, BorderLayout.SOUTH);
```

```
// Create item count label
itemCountLabel = new JLabel("Item Count: 0");
mainPanel.add(itemCountLabel, BorderLayout.NORTH);
```

```
// Add action listeners directly to the buttons
addItemButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        addItem();
    }
});
```

```
removeItemButton.addActionListener(new ActionListener() {
    @Override
```

```

        public void actionPerformed(ActionEvent e) {
            removeItem();
        }
    });

    displayDetailsButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            displayDetails();
        }
    });

    // Initialize the GUI with some sample data
    initializeSampleData();
}

private void initializeSampleData() {
    StockItem item1 = new StockItem("Iphone 14 Pro Max");
    StockItem item2 = new StockItem("Samsung Galaxy Ultra");
    StockItem item3 = new StockItem("Apple Vision");

    addItemToList(item1);
    addItemToList(item2);
    addItemToList(item3);
}

private void addItemToList(StockItem item) {
    itemsListModel.addElement(item);
    itemCount++;
    updateItemCountLabel();
}

private void removeItemFromList(StockItem item) {
    itemsListModel.removeElement(item);
    itemCount--;
    updateItemCountLabel();
}

private void addItem() {
    // Show an input dialog to get the item name from the user
    String name = JOptionPane.showInputDialog("Enter item name:");

```



```

        // Create a new StockItem object with the provided name
        StockItem newItem = new StockItem(name);

        // Add the item to the list
        addItemToList(newItem);
    }

    private void removeItem() {
        // Get the selected item from the list
        StockItem selectedItem = itemsList.getSelectedValue();

        // Remove the selected item from the list
        if (selectedItem != null) {
            removeItemFromList(selectedItem);
        } else {
            JOptionPane.showMessageDialog(this, "No item selected.");
        }
    }

    private void displayDetails() {
        // Get the selected item from the list
        StockItem selectedItem = itemsList.getSelectedValue();

        // Display the name of the item in a dialog box
        if (selectedItem != null) {
            JOptionPane.showMessageDialog(this, "Item: " + selectedItem.getName(), "Item
Details", JOptionPane.INFORMATION_MESSAGE);
        } else {
            JOptionPane.showMessageDialog(this, "No item selected.", "Error",
JOptionPane.ERROR_MESSAGE);
        }
    }

    private void updateItemCountLabel() {
        itemCountLabel.setText("Item Count: " + itemCount);
    }

    public void updateItemList(Object[] toArray) {
        itemsListModel.clear(); // Clear the existing items from the list model

        for (Object obj : toArray) {
            if (obj instanceof StockItem) {

```

```

        StockItem stockItem = (StockItem) obj;
        itemListModel.addElement(stockItem); // Add the stock item to the list model
    }
}

itemCount = itemListModel.getSize(); // Update the item count
updateItemCountLabel(); // Update the item count label
}

}

class WarehouseManagement {
    private List<StockItem> stockItems;

    public WarehouseManagement() {
        stockItems = new ArrayList<>();
    }

    public void addItem(String itemName) {
        StockItem newItem = new StockItem(itemName, 0);
        stockItems.add(newItem);
    }

    public boolean removeItem(String itemName) {
        StockItem itemToRemove = null;
        for (StockItem item : stockItems) {
            if (item.getName().equals(itemName)) {
                itemToRemove = item;
                break;
            }
        }
        if (itemToRemove != null) {
            stockItems.remove(itemToRemove);
        }
        return false;
    }

    public StockItem findStockItemByName(String itemName) {
        for (StockItem item : stockItems) {
            if (item.getName().equals(itemName)) {
                return item;
            }
        }
    }
}

```

```
        return null;
    }

    public int getStockItemCount() {
        return stockItems.size();
    }
}
```